

Clean Code

✗ Ruim: Múltiplas Responsabilidades

```
function validateAndSaveUser(data) {  
  // Validação  
  if (!data.email.includes('@')) {  
    throw new Error('Email inválido');  
  }  
  // Transformação  
  const user = { email: data.email.toLowerCase() };  
  // Salvamento  
  database.save(user);  
  emailService.send(user.email);  
}
```

✓ Bom: Uma Responsabilidade

```
function validateUser(data) {  
  if (!data.email.includes('@')) {  
    throw new Error('Email inválido');  
  }  
}
```

```
function registerUser(data) {  
  validateUser(data);  
  const user = normalizeUser(data);  
  database.save(user);  
  emailService.send(user.email);  
}
```

✗ Ruim: Muitos Parâmetros

```
function createUser(name, email, age, phone,address, city, country,  
zipCode) {  
  // ... }
```

```
createUser('João', 'joao@email.com',  
30, '123456789', 'Rua A',  
'São Paulo', 'Brasil', '01310')
```

✓ Bom: Objeto como Parâmetro

```
function createUser(userData) {  
  
  const { name, email, age, phone,  
    address, city, country } = userData;  
  // ...  
}  
  
createUser({  
  name: 'João', email: 'joao@email.com',  
  age: 30, phone: '123456789',  
  address: 'Rua A', city: 'São Paulo'  
})
```

Regra 1: Use Exceções, Não Códigos de Erro

```
function getUserSucess(id) {  
  const user = db.findById(id);  
  
  if (!user) {  
    return {  
      error: 'USER_NOT_FOUND',  
      code: 404}; }  
  return { success: true, data: user };  
}  
function getUserError(id) {  
  const result = getUser(123);  
  if (result.error) { console.log(result.error); }
```

Regra 1: Use Exceções, Não Códigos de Erro

```
function getUserSucess(id) {  
  const user = db.findById(id);
```

```
  if (!user) {  
    return {  
      error: 'USER_NOT_FOUND',  
      code: 404}; }  
  return { success: true, data: user };  
}
```

```
function getUserError(id) {  
  const result = getUser(123);  
  if (result.error) { console.log(result.error); }
```

SOLID

SOLID: Single Responsibility Principle

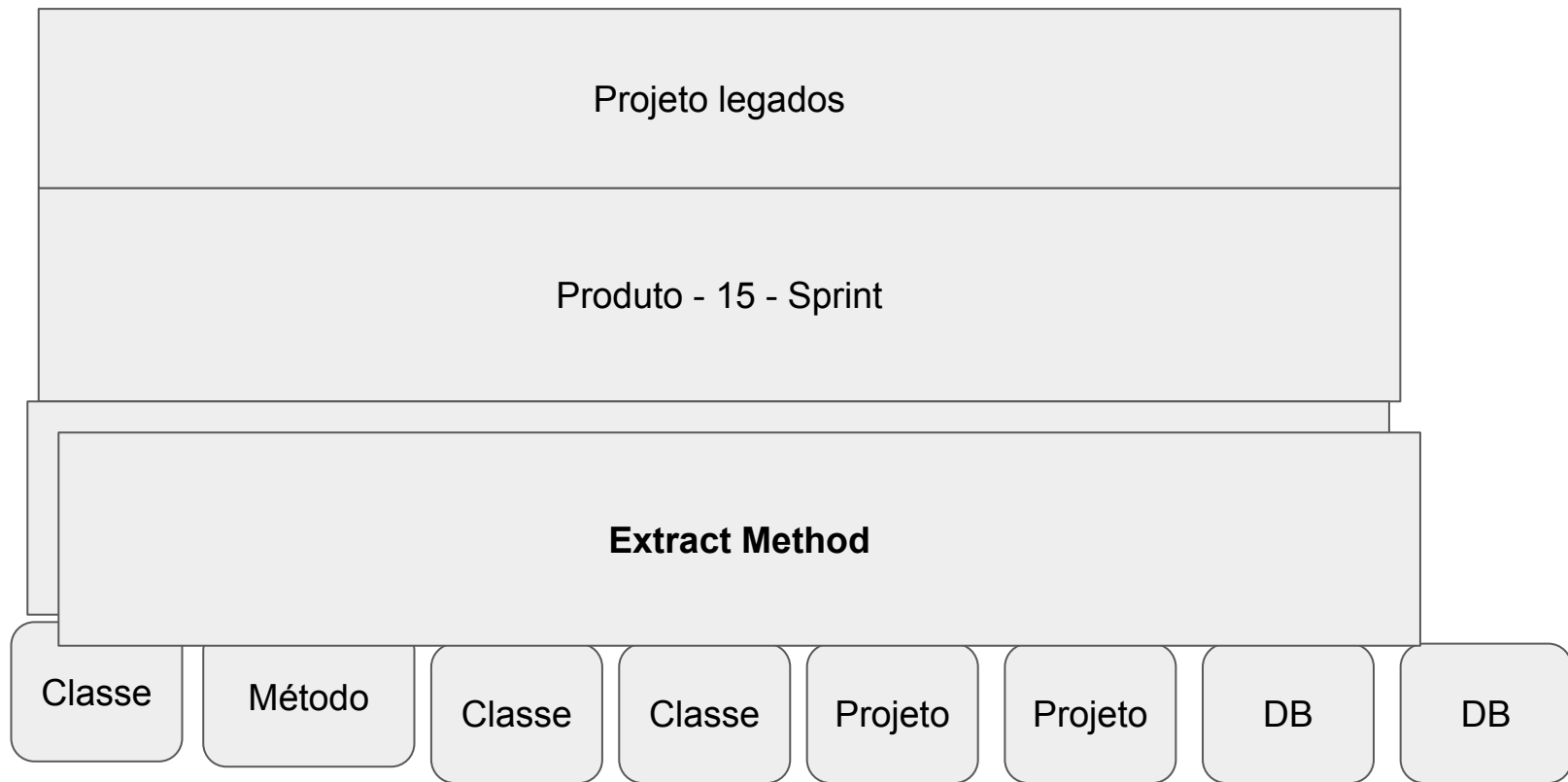
```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  getName() { return this.name; }  
  validateEmail() { /* ... */ }  
  sendEmail(msg) { /* ... */ }  
  saveToDatabase() { /* ... */ }}
```

SOLID: Open/Closed, Liskov, Interface, Dependency

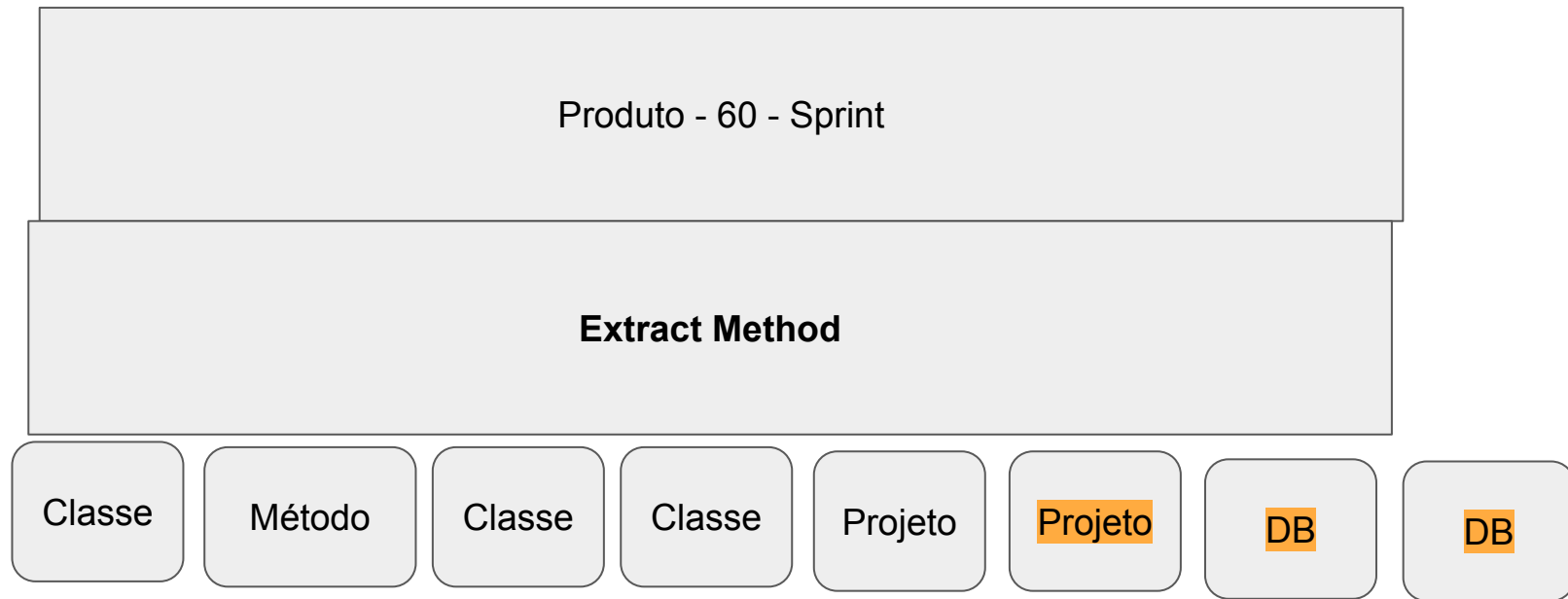
```
class PaymentProcessor {  
  processPayment(type, amt) {  
    if (type === 'credit') {  
      // Lógica cartão  
    } else if (type === 'debit') {  
      // Lógica débito  
    } else if (type === 'paypal') {  
      // Lógica PayPal  
    }  
  }  
}
```

SOLID

Refatoração



Refatoração



Refatoração Incremental

Produto - 15 - Sprint		
Refinamento Técnico - Extract Method		
Análise de Risco - 2 Sprint - 2 DEV		

Classe	Método	Classe
--------	--------	--------

Débito Técnico 1

4 H

Classe	Projeto	Projeto
--------	---------	---------

Débito Técnico 1

4 H

DB	DB
----	----

Débito Técnico 3

4 H
